

♦ 0. Avant Rails, Ruby

Pour lire du code Rails, il faut distinguer le langage du framework. `has_many :sessions`, c'est Rails ; une méthode qui fabrique d'autres méthodes au chargement de la classe, c'est Ruby. Et en Ruby, contrairement à PHP, une classe peut être rouverte et modifiée pendant l'exécution, depuis n'importe quel fichier : le code que vous lisez n'est pas forcément le code qui s'exécute (section 6).

- Toutes les méthodes d'une classe : <https://rubyapi.org/>
- La documentation officielle : <https://docs.ruby-lang.org/>
- Les supports sont écrits pour Ruby 4.0 (`ruby -v`). Votre application tourne très probablement sur des versions plus anciennes de Ruby et de Rails.

Tapez les exemples dans IRB, dans l'ordre : ils s'enchaînent.

♦ 1. Les bases : tout est objet

→ Avant de lire cette section, faites le 1.1 de `ruby/exercices/01_bases.md`, sans rien lire d'autre.

Exploration en IRB

Lancez `irb` : c'est votre terrain de jeu, et votre premier outil de lecture de code.

```
1.class
nil.class
:dog.class
{ name: "Rex" }.class
42.even?
3.times { puts "wouf" }
5.+(3)           # => 8, exactement comme 5 + 3
```

Tout est objet, même `nil`, et tout est appel de méthode, même `+`, `==`, `<<` ou `[]` sont aussi des méthodes, qu'une classe peut redéfinir.

Pour afficher, `puts` utilise `to_s` (une ligne vide pour `nil`) et renvoie `nil` ; `p` utilise `inspect` et renvoie son argument. Dans le doute, `p`.

La truthiness en Ruby

Seuls `nil` et `false` sont faux. `0`, `""` et `[]` sont vrais :

```
!!0           # => true  (!! convertit en booléen)
!![]          # => true
!!nil         # => false
```

`if` `tableau` ne teste donc pas si le tableau est vide : il faut le demander avec `empty?`. Rails ajoute `blank?` et `present?` précisément parce que `""` et `[]` ne sont pas faux.

Conditions

```
age = 4
if age < 1
  "chiot"
elsif age < 8           # elsif, en un mot
  "adulte"
else
  "senior"
end                     # => "adulte"

age > 7 ? "senior" : "adulte" # => "adulte"
puts "vacciné" unless age < 1 # unless : « si non », ici en forme suffixe
hours, minutes = 90.divmod(60) # affectation multiple : hours vaut 1, minutes 30
```

`if` renvoie une valeur, comme toute expression Ruby : `label = if ... end` est permis.

Symboles et chaînes

```
"Rex".object_id == "Rex".object_id # => false : deux chaînes distinctes
:dog.object_id == :dog.object_id    # => true : un seul symbole en mémoire
:dog.to_s                           # => "dog"
"dog".to_sym                         # => :dog
```

Un symbole est unique et immuable. Règle pratique : **une chaîne pour du contenu, un symbole pour un nom** : clé de hash, statut, nom de méthode. Dans `attr_reader :name` (section 2), `:name` est le nom de la méthode à fabriquer.

Méthodes

```
def introduce(name, species = :dog, age: nil)
  text = "#{name}, #{species}"
  text += ", #{age} ans" if age
  text
end

introduce("Rex")           # => "Rex, dog"
introduce "Felix", :cat, age: 12 # => "Felix, cat, 12 ans"
```

- **Retour implicite** : une méthode renvoie sa dernière expression, ici `text`. `return` ne sert qu'à sortir plus tôt.
- **Parenthèses facultatives** : le second appel s'en passe. `has_many :sessions` est ainsi un appel de méthode.
- **Valeur par défaut et keyword argument** : `species` vaut `:dog` s'il est omis ; `age:` se passe toujours par son nom, et `introduce("Rex", :dog, 5)` lève une `ArgumentError`.
- **Nommage** : `?` pour un booléen (`empty?`) ; `!` pour la version « dangereuse » d'une méthode, qui modifie l'objet en place, ou lève une exception là où l'autre renvoie `false`.

```
name = "rex"
name.upcase # => "REX", name vaut toujours "rex"
name.upcase! # => "REX", name a changé
```

Interpolation et hashes

```
name = "Rex"
"Bonjour #{name}"      # => "Bonjour Rex"
'Bonjour #{name}'      # => "Bonjour \#{name}" : pas d'interpolation entre guillemets simples

animal = { name: "Rex", species: :dog }      # syntaxe moderne
animal = { :name => "Rex", :species => :dog }  # syntaxe historique, même hash

animal[:name]          # => "Rex"
animal[:chip]          # => nil, sans erreur
animal["name"]         # => nil : la clé est le symbole :name, pas la chaîne "name"
animal.fetch(:chip)    # => KeyError: key not found: :chip
animal[:owner]&.upcase  # => nil : &. n'appelle upcase que sur un objet non nil
```

[] avale les fautes de frappe en silence, fetch les signale.

Une chaîne de plusieurs lignes s'écrit en heredoc, avec interpolation ; ~ retire l'indentation commune :

```
card = <<~TEXT
  Nom : #{name}
  Espèce : chien
TEXT
card   # => "Nom : Rex\nEspèce : chien\n"
```

→ Ouvrez `ruby/exercices/01_bases.md`

♦ 2. Les classes

Explorer les classes en IRB

```
class Animal
  def initialize(name)
    @name = name
  end
end

rex = Animal.new("Rex")  # IRB affiche #<Animal:0x... @name="Rex">
rex.name                 # => NoMethodError: undefined method 'name' for an instance of Animal
```

@name existe, et pourtant rex.name échoue. Le constructeur s'appelle initialize, Animal.new est un appel de méthode, et @name est une variable d'instance, sans \$this-> ni déclaration. Surtout, **il n'existe pas de propriété publique en Ruby** : on passe toujours par une méthode.

Il faut donc les écrire. Une seconde déclaration class Animal complète la classe existante au lieu de la remplacer (section 6) :

```
class Animal
  def name          # lire @name
    @name
  end

  def name=(new_name)  # modifier @name
    @name = new_name
  end
end

rex.name           # => "Rex"
rex.name = "Médor" # appelle la méthode name=, comme rex.name=("Médor")
rex.name           # => "Médor"
```

attr_reader, attr_writer, attr_accessor

Le raccourci : `attr_reader :status` génère `status`, `attr_writer :status` génère `status=`, `attr_accessor :status` les deux.

```
class Animal
  attr_accessor :status
end

Animal.instance_methods(false).sort # => [:name, :name=, :status, :status=]
```

`attr_accessor` n'est pas un mot-clé mais une **méthode**, appelée sur la classe pendant sa lecture, qui **fabrique** d'autres méthodes : `:status` existe sans aucun `def`. `instance_methods(false)` liste les méthodes publiques définies par la classe elle-même.

self et le receveur implicite, méthodes de classe et visibilité

```
class Animal
  def self.species          # méthode de classe, le static de PHP
    [:dog, :cat, :fish]
  end

  def fall_asleep
    self.status = :asleep  # self obligatoire pour appeler un setter
    "#{name} fait #{snore}" # name tout seul : self.name
  end

  private

  def snore
    "zzz"
  end
end

Animal.species # => [:dog, :cat, :fish]
rex.fall_asleep # => "Médor fait zzz"
rex.status      # => :asleep
rex.snore       # => NoMethodError: private method 'snore' called for an instance of Animal
```

Sans receveur écrit, une méthode est appelée sur `self`, l'objet courant. Le piège : sans `self.`, `status = :asleep` crée une variable locale, sans erreur, et le statut ne change pas. Hors de toute classe, `self` est `main`, l'objet de premier niveau : un appel sans receveur qui échoue y signale `undefined method '...' for main`.

`private` est lui aussi un appel de méthode : il rend privées les méthodes définies après lui. Enfin, `puts rex` appelle `rex.to_s` : redéfinissez `to_s` pour un affichage lisible.

→ Ouvrez [ruby/exercices/02_classes.md](#)

◆ 3. Héritage et modules

Héritage

```
class Vehicle
  attr_reader :name, :wheels

  def initialize(name, wheels)
    @name = name
    @wheels = wheels
  end
end

class Bike < Vehicle
  def initialize(name)
    super(name, 2)
  end
end

Bike.new("VTT").wheels # => 2
```

`super` appelle la méthode du même nom chez le parent : `super(name, 2)` avec ces arguments, `super()` sans aucun, et `super` seul, **sans parenthèses**, avec tous les arguments reçus. `class Workshop < ApplicationRecord` hérite ainsi de tout `ActiveRecord`.

Modules : les mixins

Un module partage des méthodes entre plusieurs classes, sans héritage multiple :

```
module Insurable
  def annual_premium
    power * 12
  end
end

class Car < Vehicle
  include Insurable
  attr_reader :power

  def initialize(name, power = 7)
    super(name, 4)
    @power = power
  end
end

Car.new("Zoé").annual_premium # => 84
```

`Insurable` appelle `power` sans le définir : un **contrat implicite**, qu'aucune `interface` n'écrit. Si la classe ne fournit pas `power`, l'erreur n'arrive qu'à l'appel. Ruby en a aussi : `Enumerable` exige `each`, `Comparable` exige `<=>`. Les concerns de Rails (`app/models/concerns/`) sont de tels modules.

extend : des méthodes pour un seul objet

```
module Countable
  def counter
    @counter ||= 0      # affecte seulement si @counter vaut nil ou false
  end
end

Car.extend(Countable)  # dans le corps de la classe : extend Countable
Car.counter            # => 0
Car.new("Zoé").counter # => NoMethodError
```

`include` donne les méthodes du module aux instances ; `extend` les donne à un seul objet. Une classe étant un objet, `extend` lui donne des méthodes de classe.

Modules comme espaces de noms

Un module range aussi des classes, comme un namespace PHP : après `module Garage; class Car; end; end`, `Garage::Car` est une autre classe que `Car`. Même notation pour les constantes, dont le nom commence par une majuscule (un nom de classe en est une) : `SPECIES = %i[dog cat fish].freeze`, écrit dans `Animal`, se lit `Animal::SPECIES` depuis l'extérieur.

ancestors : où Ruby va chercher

```
Car.ancestors # => [Car, Insurable, Vehicle, Object, PP::ObjectMixin, Kernel, BasicObject]
```

L'ordre dans lequel Ruby cherche une méthode appelée sur une voiture. Un module inclus se place juste après la classe ; avec `prepend`, il se place avant (section 6). `PP::ObjectMixin` vient d'IRB, qui charge la bibliothèque `pp` : un script lancé avec `ruby` ne l'affiche pas.

Duck typing

Ruby ne vérifie pas les types : on ne demande pas à l'objet s'il est une voiture, mais s'il sait répondre.

```
[Car.new("Zoé"), Bike.new("VTT"), "une chaîne"].each do |object|
  puts object.annual_premium if object.respond_to?(:annual_premium)
end
# 84 : seule la voiture sait répondre
```

→ Ouvrez `ruby/exercices/03_heritage_modules.md`

♦ 4. Blocs et itérateurs

Les blocs

Un bloc est un morceau de code passé à une méthode, en plus de ses paramètres : `{ |n| ... }` sur une ligne, `do |n| ... end` sur plusieurs. Vous en avez déjà écrit : `3.times { puts "wouf" }`.

Un bloc voit et modifie les variables locales qui l'entourent ; un `def`, lui, ne voit que ses paramètres :

```
total = 0
[3, 5, 2].each { |n| total += n }
total      # => 10
```

Écrire une méthode qui accepte un bloc

`yield` exécute le bloc et renvoie sa valeur :

```
def timed
  start = Time.now
  result = yield
  puts "Durée : #{Time.now - start} s"
  result
end

timed { (1..1_000_000).sum } # => 500000500000
```

`yield(x)` passe `x` au bloc. Sans bloc, `yield` lève une `LocalJumpError` : testez avec `block_given?`. Et `&` dans la signature range le bloc dans un paramètre, pour le **transmettre** à une autre méthode.

```
def species
  return [:dog, :cat, :fish] unless block_given?

  [:dog, :cat, :fish].each { |kind| yield(kind) }
end

species # => [:dog, :cat, :fish]
species { |kind| puts "Espèce : #{kind}" }

def filter(names, &criterion) # le bloc devient un objet...
  names.select(&criterion) # ... transmis à select
end

filter(["Rex", "Felix", "Médor"]) { |name| name.start_with?("M") } # => ["Médor"]
```

Enumerable, les itérateurs essentiels

```
animals = [
  { name: "Rex", species: :dog, age: 5 },
  { name: "Felix", species: :cat, age: 12 },
  { name: "Bulle", species: :fish, age: 2 },
  { name: "Médor", species: :dog, age: 8 }
]

animals.map { |a| a[:name] } # => ["Rex", "Felix", "Bulle", "Médor"]
animals.select { |a| a[:species] == :dog } # garder : Rex et Médor
animals.reject { |a| a[:species] == :fish } # écarter
animals.find { |a| a[:species] == :cat } # le premier qui correspond, ou nil
animals.count { |a| a[:age] > 4 } # => 3
animals.sum { |a| a[:age] } # => 27
animals.sort_by { |a| a[:age] } # trier
animals.min_by { |a| a[:age] } # le plus jeune : Bulle
animals.map { |a| a[:species] }.tally # => {dog: 2, cat: 1, fish: 1}
animals.flat_map { |a| [a[:name], a[:name].upcase] } # map, puis aplatit d'un niveau
```

`each` renvoie le tableau d'origine, le résultat du bloc est perdu ; `map` renvoie un tableau des résultats.

`group_by` renvoie un hash. Sur un hash, `each` passe au bloc la clé et la valeur :

```
animals.group_by { |a| a[:species] }.each do |species, list|
  puts "#{species} : #{list.map { |a| a[:name] }.join(", ")}"
end
# dog : Rex, Médor
# cat : Felix
# fish : Bulle
```

`reduce` et `each_with_object` savent tout faire. Voici `sum` et `tally` écrits à la main (`Hash.new(0)` : les clés absentes valent 0) :

```
animals.reduce(0) { |total, a| total + a[:age] } # => 27

animals.each_with_object(Hash.new(0)) do |a, counter|
  counter[a[:species]] += 1
end
# => {dog: 2, cat: 1, fish: 1}
```

Chaîner

```
# Les noms des chiens de plus de 3 ans, par ordre alphabétique
animals.select { |a| a[:species] == :dog }.select { |a| a[:age] > 3 }.map { |a| a[:name] }.sort
# => ["Médor", "Rex"]
```

Dans un fichier, on met souvent un appel par ligne, le point en début de ligne. Dans IRB, gardez la chaîne sur une ligne : IRB exécute chaque ligne complète.

&:symbole et to_proc

```
["rex", "felix"].map(&:upcase) # => ["REX", "FELIX"]
```

`&:upcase` équivaut à `{ |x| x.upcase }` : `&` appelle `to_proc` sur le symbole, qui fabrique ce bloc.

Enumerable dans votre propre classe

Toutes ces méthodes, hors `each`, viennent du module `Enumerable`. Son contrat implicite : que la classe définisse `each`.

```
class Fleet
  include Enumerable

  def initialize(vehicles)
    @vehicles = vehicles
  end

  def each(&block)
    @vehicles.each(&block)
  end
end

fleet = Fleet.new([Car.new("Zoé"), Bike.new("VTT"), Car.new("Clio")])
fleet.count # => 3
fleet.select { |v| v.wheels == 4 }.map(&:name) # => ["Zoé", "Clio"]
```

Pour une méthode écrite, la classe en gagne une soixantaine. C'est aussi pourquoi une relation Active Record se parcourt avec `each`, `map` ou `select`, comme un tableau.

Proc et lambda

Un bloc rangé dans une variable est un `Proc`. Une lambda est un `Proc` strict sur le nombre d'arguments :

```
double = ->(n) { n * 2 }      # lambda, s'écrit aussi lambda { |n| n * 2 }
double.call(5)                # => 10
double.call(5, 6)             # => ArgumentError: wrong number of arguments (given 2, expected 1)

twice = Proc.new { |n| n * 2 } # Proc, comme tout bloc
twice.call(5, 6)              # => 10 : argument en trop ignoré (un argument manquant vaudrait nil)
```

Dans un modèle Rails, `scope :published, -> { where(published: true) }` : la lambda fait exécuter la requête à chaque appel du scope, et non une seule fois au chargement de la classe.

→ Ouvrez `ruby/exercices/04_iterateurs.md`

♦ 5. Exceptions

Lever et attraper

```
def feed(animal, grams)
  raise ArgumentError, "quantité invalide : #{grams}" unless grams.positive?

  "#{animal} a mangé #{grams} g"
end

begin
  feed("Rex", -10)
rescue ArgumentError => e
  puts "Erreur : #{e.message}"
else
  puts "Tout s'est bien passé" # seulement si aucune exception
ensure
  puts "Gamelle rangée"       # toujours, exception ou pas
end
```

Dans un `def`, le `begin` est facultatif :

```
def feed_safely(animal, grams)
  feed(animal, grams)
rescue ArgumentError
  "#{animal} n'a rien eu"
end

feed_safely("Rex", -10) # => "Rex n'a rien eu"
```

`retry`, dans un `rescue`, relance le `begin` depuis le début : pratique pour un appel réseau, dangereux sans compteur de tentatives.

Vos propres erreurs

Une exception est une classe : une hiérarchie d'erreurs permet d'attraper large ou précis, et chaque erreur peut porter ses données.

```
class VetError < StandardError; end

class UnknownAnimalError < VetError
  attr_reader :name

  def initialize(name)
    @name = name
    super("aucun animal nommé #{name}")
  end
end

begin
  raise UnknownAnimalError.new("Rantanplan")
rescue VetError => e      # attrape aussi toutes les filles de VetError
  puts "#{e.class} : #{e.message}" # UnknownAnimalError : aucun animal nommé Rantanplan
  puts e.name              # Rantanplan
end
```

Les clauses `rescue` sont testées dans l'ordre : la plus précise en premier.

Ce que `rescue` attrape vraiment

`rescue => e`, sans classe, attrape les `StandardError`. `Exception`, au-dessus, contient aussi `Interrupt` (Ctrl+C) et `SystemExit` (`exit`) : **jamais de `rescue Exception`**. Même `rescue => e` est souvent trop large, puisqu'il attrape la `NoMethodError` d'une faute de frappe : visez la classe la plus précise.

La convention méthode / méthode!

Pour une action qui peut échouer, beaucoup de bibliothèques offrent deux versions : sans `!`, elle renvoie `false` ; avec `!`, elle lève une exception. En Rails : `save` et `save!`.

→ Ouvrez `ruby/exercices/05_exceptions.md`

Une question à garder. Dans IRB, tapez `String.ancestors`. Vous n'avez jamais écrit `include Comparable` dans `String` : d'où vient-il ? Et `Kernel` ? Pourquoi `puts` marche-t-il n'importe où ? Réponse en section 6.

♦ 6. Métaprogrammation et introspection

→ Avant de lire cette section, faites la partie libre du **6.1** de `ruby/exercices/06_metaprogrammation.md` (jusqu'à « Maintenant, avec méthode »), 15 minutes, sans aide.

Vous écrierez rarement de la métaprogrammation, mais vous en lirez beaucoup. Cette section donne les outils pour retrouver d'où vient une méthode écrite nulle part.

Les exemples prolongent ceux des sections précédentes. Dans un nouvel IRB, retapez d'abord `Vehicle`, `Insurable` et `Car` depuis la section 3.

La chaîne de résolution des méthodes

Pour exécuter `car.start`, Ruby parcourt une liste d'endroits, dans un ordre précis, et exécute la première méthode `start` trouvée.

```
module Electric
  def start
    "silence... " + super
  end
end

class Car
  prepend Electric

  def start
    "vroum"
  end
end

Car.new("Zoé").start # => "silence... vroum"
Car.ancestors
# => [Electric, Car, Insurable, Vehicle, Object, PP::ObjectMixin, Kernel, BasicObject]
```

`car.start` : Ruby cherche de haut en bas

singleton de car	méthodes ajoutées à CET objet seul (<code>car.extend(Module)</code>)
Electric	prepend : AVANT la classe, peut l'envelopper avec <code>super</code>
Car	la classe elle-même, ce que vous lisez dans son fichier
Insurable	include : APRÈS la classe
Vehicle	la classe parente, puis ses propres modules...
Object, Kernel, BasicObject	
rien trouvé	Ruby re-parcourt la même liste en cherchant <code>method_missing</code>

Avec `include`, la classe gagne sur le module et atteint sa version par `super`. Avec `prepend`, le module gagne sur la classe : vous lisez `Car#start`, c'est `Electric#start` qui s'exécute. `extend` ajoute le module à un seul objet, dans sa `singleton_class` : sur une classe, cela donne des méthodes de classe, comme `def self..`

La réponse à la question de la section 5 : `String` inclut `Comparable` dans le code de Ruby lui-même, et `Object` inclut `Kernel`, qui porte `puts`, `require`, `raise` ... Tout objet hérite d'`Object` : `puts` est partout. `PP::ObjectMixin`, lui, est inclus dans `Object` par la bibliothèque `pp` que charge IRB.

Classes ouvertes

En PHP, déclarer deux fois une classe est une erreur fatale. En Ruby, la seconde déclaration **rouvre** la classe :

```
class String
  def shout
    upcase + " !"
  end
end

"attention".shout # => "ATTENTION !"
```

Toutes les chaînes du programme ont maintenant `shout`. Rails en use largement : `"".blank?` et `3.days.ago` sont des ajouts de Rails aux classes de Ruby. Le revers : n'importe quel fichier chargé peut remplacer une méthode, et si deux fichiers définissent la même, le dernier chargé gagne, sans erreur.

La boîte à outils du lecteur de code

Le fichier ne dit pas tout : on interroge le programme qui tourne, dans IRB ou dans la console Rails.

Besoin	Outil
Que sait faire cet objet ?	<code>ls car</code> (commande IRB)
Quel est le code de cette méthode ?	<code>show_source Car#start</code> (commande IRB)
D'où vient cette méthode ?	<code>car.method(:start).owner</code>
Dans quel fichier, à quelle ligne ?	<code>car.method(:start).source_location</code>
Dans quel ordre Ruby cherche-t-il ?	<code>Car.ancestors</code>
Que définit la classe elle-même ?	<code>Car.instance_methods(false)</code>
L'objet sait-il répondre à cette méthode ?	<code>car.respond_to?(:start)</code>
Qui a appelé ce code ?	<code>caller</code>
Je veux m'arrêter ici et regarder	<code>binding.irb</code>

```
car = Car.new("Zoé")
ls car
# Electric#methods: start
# Car#methods: power
# Insurable#methods: annual_premium
# Vehicle#methods: name wheels
# instance variables: @name @power @wheels

car.method(:start).owner          # => Electric
car.method(:start).source_location # ["(irb)", numéro de ligne] : fichier et ligne du def
car.method(:start).super_method  # Car#start : la méthode que super appellera
car.method(:annual_premium).owner # => Insurable
"x".method(:upcase).source_location # => nil : méthode écrite en C, dans Ruby lui-même
```

`ls` range les méthodes par module d'origine ; `show_source Car#start` affiche `Electric#start`, le code qui s'exécute.

- `source_location` vaut `nil` : méthode écrite en C. Un fichier du projet pour une méthode de Ruby comme `String#upcase` : quelqu'un l'a redéfinie.
- `source_location` pointe vers une ligne qui n'est pas un `def` : la méthode a été **fabriquée** par une autre. Vous êtes dans le générateur, pas dans la déclaration.
- `binding.irb`, posé dans le code, ouvre une console à cet endroit, avec les variables locales et `self` ; `exit` reprend l'exécution.

send et public_send

```
action = :start
car.send(action)           # => "silence... vroum"
car.public_send("sta" + "rt") # => "silence... vroum"
```

Le nom de la méthode peut n'apparaître **nulle part** en clair : un grep sur `start` ne trouve pas le second appel. `send` peut appeler une méthode privée, `public_send` respecte la visibilité.

define_method : fabriquer des méthodes

```
class Vehicle
  [:petrol, :diesel, :electric].each do |fuel|
    define_method("#{fuel}?") do
      @fuel == fuel
    end
  end
end

Vehicle.instance_methods(false).sort # => [:diesel?, :electric?, :name, :petrol?, :wheels]
car.instance_variable_get("@name")   # => "Zoé"
```

Trois méthodes, aucun `def` : c'est le principe d'attr_accessor, de has_many, d'enum, et de tout générateur de code. De même, `instance_variable_get` et `instance_variable_set` accèdent à une variable d'instance dont le nom est calculé.

method_missing et respond_to_missing?

Quand Ruby ne trouve la méthode nulle part dans `ancestors`, il appelle `method_missing`, qui lève par défaut la `NoMethodError`. C'est le `__call__` de PHP :

```
class Fleet
  def initialize(vehicles)
    @vehicles = vehicles
  end

  def method_missing(name, *args, &block)
    if name.to_s.start_with?("vehicles_with_")
      wheels = name.to_s.delete_prefix("vehicles_with_").to_i
      @vehicles.select { |v| v.wheels == wheels }
    else
      super
    end
  end

  def respond_to_missing?(name, include_private = false)
    name.to_s.start_with?("vehicles_with_") || super
  end
end

fleet = Fleet.new([Car.new("Zoé"), Vehicle.new("VTT", 2), Car.new("Clio")])
fleet.vehicles_with_4.map(&:name) # => ["Zoé", "Clio"]
fleet.respond_to?(:vehicles_with_2) # => true
```

Deux règles : finir par `super`, sinon toute faute de frappe devient silencieuse ; écrire `respond_to_missing?`, sinon `respond_to?` ment. Active Record s'en sert encore : `Workshop.find_by_title("...")` n'est écrit nulle part, `method_missing` le traite.

Les hooks : du code qui s'exécute au chargement

Ruby prévient un module quand une classe l'inclut :

```
module Licensable
  def self.included(base)      # appelé quand une classe fait include Licensable
    base.extend(ClassMethods)
  end

  module ClassMethods
    def plate_prefix(prefix)
      @plate_prefix = prefix
    end
  end
end

class Car
  include Licensable
  plate_prefix "AB"           # une méthode de classe apportée par l'include
end
```

Avec ce motif, un seul `include` apporte des méthodes d'instance et des méthodes de classe. Rails l'habille avec `extend ActiveSupport::Concern`, qui offre `included do ... end` et `class_methods do ... end`. Une simple ligne `include` peut donc exécuter du code, ajouter des méthodes de classe, et même faire un `prepend`.

class_eval et instance_eval

```
Car.class_eval do
  def honk
    "tut"
  end
end

{ front: "avant", rear: "arrière" }.each do |side, label|
  Car.class_eval <<~RUBY, __FILE__, __LINE__ + 1
    def #{side}_lights
      "feux #{label} allumés"
    end
  RUBY
end

Car.new("Zoé").rear_lights      # => "feux arrière allumés"
Car.new("Zoé").instance_eval { @power }  # => 7
```

`class_eval` ouvre une classe tenue dans une variable. Avec une **chaîne**, c'est la forme la plus opaque : le code n'existe qu'une fois la chaîne assemblée, aucun fichier ne contient `def rear_lights`. `instance_eval` exécute un bloc avec `self` égal à un objet, d'où l'accès à ses variables d'instance : c'est le principe des DSL comme `Rails.application.routes.draw do ... end`.

Pourquoi du code généré ressemble à ça

Un humain qui écrit dix modèles écrit dix fois `def title`. Un générateur factorise : une méthode qui fabrique les accesseurs, puis une ligne de déclaration par modèle. Le résultat est cohérent et systématique, mais les méthodes n'apparaissent plus en clair : elles sont construites au chargement, par `define_method`, `method_missing` ou des chaînes évaluées. Ce code reste lisible avec la boîte à outils.

→ Ouvrez `ruby/exercices/06_metaprogrammation.md`

♦ 7. Synthèse (bonus) et organisation en fichiers

Charger du code

Dès que le code grossit, on le découpe en fichiers, que l'on charge ainsi :

```
require_relative "vehicle" # charge ./vehicle.rb, relatif au fichier courant
require "json"             # cherche json.rb dans les dossiers de $LOAD_PATH
```

Les deux ne chargent un fichier qu'une seule fois. L'ordre compte : `include Insurable` exige qu' `Insurable` soit déjà chargé. En Rails, presque plus de `require` : Zeitwerk déduit le fichier du nom de la constante (step 02).

Gems et Bundler

Une gem est une bibliothèque Ruby empaquetée (`gem install`). Dans un projet :

- le `Gemfile` déclare les gems et les versions acceptées ;
- `bundle install` résout les versions et les installe ;
- le `Gemfile.lock` fige les versions exactes réellement utilisées : il se versionne ;
- `bundle exec commande` lance la commande avec exactement ces versions.

Face à une application existante, le `Gemfile.lock` renseigne sur la version de Rails, les gems propriétaires, la base de données, l'âge des dépendances.

→ Ouvrez `ruby/exercices/07_synthese.md`

♦ Pour aller plus loin : la documentation officielle

Exercice 1 - Les bases de Ruby

- [Hash literals](https://docs.ruby-lang.org/en/4.0/syntax/literals_rdoc.html#label-Hash+Literals)
docs.ruby-lang.org/en/4.0/syntax/literals_rdoc.html#label-Hash+Literals
- [Return values](https://docs.ruby-lang.org/en/4.0/syntax/methods_rdoc.html#label-Return+Values)
docs.ruby-lang.org/en/4.0/syntax/methods_rdoc.html#label-Return+Values
- [NoMethodError](https://docs.ruby-lang.org/en/4.0/NoMethodError.html)
docs.ruby-lang.org/en/4.0/NoMethodError.html
- [String literals](https://docs.ruby-lang.org/en/4.0/syntax/literals_rdoc.html#label-String+Literals)
docs.ruby-lang.org/en/4.0/syntax/literals_rdoc.html#label-String+Literals
- [NilClass#to_s](https://docs.ruby-lang.org/en/4.0/NilClass.html#method-i-to_s)
docs.ruby-lang.org/en/4.0/NilClass.html#method-i-to_s
- [Symbol](https://docs.ruby-lang.org/en/4.0/Symbol.html)
docs.ruby-lang.org/en/4.0/Symbol.html
- [TrueClass](https://docs.ruby-lang.org/en/4.0/TrueClass.html)
docs.ruby-lang.org/en/4.0/TrueClass.html
- [String#length](https://docs.ruby-lang.org/en/4.0/String.html#method-i-length)
docs.ruby-lang.org/en/4.0/String.html#method-i-length
- [String#upcase!](https://docs.ruby-lang.org/en/4.0/String.html#method-i-upcase-21)
docs.ruby-lang.org/en/4.0/String.html#method-i-upcase-21
- [Hash#fetch](https://docs.ruby-lang.org/en/4.0/Hash.html#method-i-fetch)
docs.ruby-lang.org/en/4.0/Hash.html#method-i-fetch
- [Hash: Key not found?](https://docs.ruby-lang.org/en/4.0/Hash.html#class-Hash-label-Key+Not+Found-3F)
docs.ruby-lang.org/en/4.0/Hash.html#class-Hash-label-Key+Not+Found-3F
- [Safe navigation operator](https://docs.ruby-lang.org/en/4.0/syntax/calling_methods_rdoc.html#label-Safe+Navigation+Operator)
docs.ruby-lang.org/en/4.0/syntax/calling_methods_rdoc.html#label-Safe+Navigation+Operator
- [Keyword arguments](https://docs.ruby-lang.org/en/4.0/syntax/methods_rdoc.html#label-Keyword+Arguments)
docs.ruby-lang.org/en/4.0/syntax/methods_rdoc.html#label-Keyword+Arguments
- [ArgumentError](https://docs.ruby-lang.org/en/4.0/ArgumentError.html)
docs.ruby-lang.org/en/4.0/ArgumentError.html
- [Ternary if](https://docs.ruby-lang.org/en/4.0/syntax/control_expressions_rdoc.html#label-Ternary+if)
docs.ruby-lang.org/en/4.0/syntax/control_expressions_rdoc.html#label-Ternary+if
- [Method names](https://docs.ruby-lang.org/en/4.0/syntax/methods_rdoc.html#label-Method+Names)
docs.ruby-lang.org/en/4.0/syntax/methods_rdoc.html#label-Method+Names

- [Logical operators](https://docs.ruby-lang.org/en/4.0/syntax/operators_rdoc.html#label-Logical+Operators)
docs.ruby-lang.org/en/4.0/syntax/operators_rdoc.html#label-Logical+Operators
- [Boolean and nil literals](https://docs.ruby-lang.org/en/4.0/syntax/literals_rdoc.html#label-Boolean+and+Nil+Literals)
docs.ruby-lang.org/en/4.0/syntax/literals_rdoc.html#label-Boolean+and+Nil+Literals
- [Modifier if and unless](https://docs.ruby-lang.org/en/4.0/syntax/control_expressions_rdoc.html#label-Modifier+if+and+unless)
docs.ruby-lang.org/en/4.0/syntax/control_expressions_rdoc.html#label-Modifier+if+and+unless
- [Array#any?](https://docs.ruby-lang.org/en/4.0/Array.html#method-i-any-3F)
docs.ruby-lang.org/en/4.0/Array.html#method-i-any-3F
- [Array#each](https://docs.ruby-lang.org/en/4.0/Array.html#method-i-each)
docs.ruby-lang.org/en/4.0/Array.html#method-i-each
- [Block argument](https://docs.ruby-lang.org/en/4.0/syntax/calling_methods_rdoc.html#label-Block+Argument)
docs.ruby-lang.org/en/4.0/syntax/calling_methods_rdoc.html#label-Block+Argument
- [Array#<<](https://docs.ruby-lang.org/en/4.0/Array.html#method-i-3C3C)
docs.ruby-lang.org/en/4.0/Array.html#method-i-3C3C

Exercice 2 - Les classes

- [Class#new](https://docs.ruby-lang.org/en/4.0/Class.html#method-i-new)
docs.ruby-lang.org/en/4.0/Class.html#method-i-new
- [Instance variables](https://docs.ruby-lang.org/en/4.0/syntax/assignment_rdoc.html#label-Instance+Variables)
docs.ruby-lang.org/en/4.0/syntax/assignment_rdoc.html#label-Instance+Variables
- [Module#attr_reader](https://docs.ruby-lang.org/en/4.0/Module.html#method-i-attr_reader)
docs.ruby-lang.org/en/4.0/Module.html#method-i-attr_reader
- [Module#attr_accessor](https://docs.ruby-lang.org/en/4.0/Module.html#method-i-attr_accessor)
docs.ruby-lang.org/en/4.0/Module.html#method-i-attr_accessor
- [Module#instance_methods](https://docs.ruby-lang.org/en/4.0/Module.html#method-i-instance_methods)
docs.ruby-lang.org/en/4.0/Module.html#method-i-instance_methods
- [Module#instance_method](https://docs.ruby-lang.org/en/4.0/Module.html#method-i-instance_method)
docs.ruby-lang.org/en/4.0/Module.html#method-i-instance_method
- [UnboundMethod#source_location](https://docs.ruby-lang.org/en/4.0/UnboundMethod.html#method-i-source_location)
docs.ruby-lang.org/en/4.0/UnboundMethod.html#method-i-source_location
- [Receiver](https://docs.ruby-lang.org/en/4.0/syntax/calling_methods_rdoc.html#label-Receiver)
docs.ruby-lang.org/en/4.0/syntax/calling_methods_rdoc.html#label-Receiver
- [Assignment methods](https://docs.ruby-lang.org/en/4.0/syntax/assignment_rdoc.html#label-Assignment+Methods)
docs.ruby-lang.org/en/4.0/syntax/assignment_rdoc.html#label-Assignment+Methods
- [Visibility](https://docs.ruby-lang.org/en/4.0/syntax/modules_and_classes_rdoc.html#label-Visibility)
docs.ruby-lang.org/en/4.0/syntax/modules_and_classes_rdoc.html#label-Visibility
- [Object#to_s](https://docs.ruby-lang.org/en/4.0/Object.html#method-i-to_s)
docs.ruby-lang.org/en/4.0/Object.html#method-i-to_s
- [Method scope \(def self.\)](https://docs.ruby-lang.org/en/4.0/syntax/methods_rdoc.html#label-Scope)
docs.ruby-lang.org/en/4.0/syntax/methods_rdoc.html#label-Scope
- [Kernel#class](https://docs.ruby-lang.org/en/4.0/Kernel.html#method-i-class)
docs.ruby-lang.org/en/4.0/Kernel.html#method-i-class
- [Integer#divmod](https://docs.ruby-lang.org/en/4.0/Integer.html#method-i-divmod)
docs.ruby-lang.org/en/4.0/Integer.html#method-i-divmod
- [String#rjust](https://docs.ruby-lang.org/en/4.0/String.html#method-i-rjust)
docs.ruby-lang.org/en/4.0/String.html#method-i-rjust
- [self](https://docs.ruby-lang.org/en/4.0/syntax/modules_and_classes_rdoc.html#label-self)
docs.ruby-lang.org/en/4.0/syntax/modules_and_classes_rdoc.html#label-self
- [Time#strftime](https://docs.ruby-lang.org/en/4.0/Time.html#method-i-strftime)
docs.ruby-lang.org/en/4.0/Time.html#method-i-strftime

Exercice 3 - Héritage et modules

- [Inheritance](https://docs.ruby-lang.org/en/4.0/syntax/modules_and_classes_rdoc.html#label-Inheritance)
docs.ruby-lang.org/en/4.0/syntax/modules_and_classes_rdoc.html#label-Inheritance
- [Method lookup](https://docs.ruby-lang.org/en/4.0/syntax/calling_methods_rdoc.html#label-Method+Lookup)
docs.ruby-lang.org/en/4.0/syntax/calling_methods_rdoc.html#label-Method+Lookup
- [Method#owner](https://docs.ruby-lang.org/en/4.0/Method.html#method-i-owner)
docs.ruby-lang.org/en/4.0/Method.html#method-i-owner
- [Method#super_method](https://docs.ruby-lang.org/en/4.0/Method.html#method-i-super_method)
docs.ruby-lang.org/en/4.0/Method.html#method-i-super_method
- [Here document literals](https://docs.ruby-lang.org/en/4.0/syntax/literals_rdoc.html#label-Here+Document+Literals)
docs.ruby-lang.org/en/4.0/syntax/literals_rdoc.html#label-Here+Document+Literals

- [String#*](https://docs.ruby-lang.org/en/4.0/String.html#method-i-2A)
docs.ruby-lang.org/en/4.0/String.html#method-i-2A
- [Object#extend](https://docs.ruby-lang.org/en/4.0/Object.html#method-i-extend)
docs.ruby-lang.org/en/4.0/Object.html#method-i-extend
- [Module#ancestors](https://docs.ruby-lang.org/en/4.0/Module.html#method-i-ancestors)
docs.ruby-lang.org/en/4.0/Module.html#method-i-ancestors
- [Object#is_a?](https://docs.ruby-lang.org/en/4.0/Object.html#method-i-is_a?)
docs.ruby-lang.org/en/4.0/Object.html#method-i-is_a-3F
- [Object#respond_to?](https://docs.ruby-lang.org/en/4.0/Object.html#method-i-respond_to?)
docs.ruby-lang.org/en/4.0/Object.html#method-i-respond_to-3F

Exercice 4 - Blocs, itérateurs et Enumerable

- [Enumerable#map](https://docs.ruby-lang.org/en/4.0/Enumerable.html#method-i-map)
docs.ruby-lang.org/en/4.0/Enumerable.html#method-i-map
- [Enumerable#find](https://docs.ruby-lang.org/en/4.0/Enumerable.html#method-i-find)
docs.ruby-lang.org/en/4.0/Enumerable.html#method-i-find
- [Enumerable](https://docs.ruby-lang.org/en/4.0/Enumerable.html)
docs.ruby-lang.org/en/4.0/Enumerable.html
- [Enumerable#each_with_object](https://docs.ruby-lang.org/en/4.0/Enumerable.html#method-i-each_with_object)
docs.ruby-lang.org/en/4.0/Enumerable.html#method-i-each_with_object
- [Hash.new](https://docs.ruby-lang.org/en/4.0/Hash.html#method-c-new)
docs.ruby-lang.org/en/4.0/Hash.html#method-c-new
- [Enumerable#group_by](https://docs.ruby-lang.org/en/4.0/Enumerable.html#method-i-group_by)
docs.ruby-lang.org/en/4.0/Enumerable.html#method-i-group_by
- [Hash#each](https://docs.ruby-lang.org/en/4.0/Hash.html#method-i-each)
docs.ruby-lang.org/en/4.0/Hash.html#method-i-each
- [Chaining method calls](https://docs.ruby-lang.org/en/4.0/syntax/calling_methods_rdoc.html#label-Chaining+Method+Calls)
docs.ruby-lang.org/en/4.0/syntax/calling_methods_rdoc.html#label-Chaining+Method+Calls
- [Hash#select](https://docs.ruby-lang.org/en/4.0/Hash.html#method-i-select)
docs.ruby-lang.org/en/4.0/Hash.html#method-i-select
- [Enumerable#flat_map](https://docs.ruby-lang.org/en/4.0/Enumerable.html#method-i-flat_map)
docs.ruby-lang.org/en/4.0/Enumerable.html#method-i-flat_map
- [Enumerable#uniq](https://docs.ruby-lang.org/en/4.0/Enumerable.html#method-i-uniq)
docs.ruby-lang.org/en/4.0/Enumerable.html#method-i-uniq
- [Block argument](https://docs.ruby-lang.org/en/4.0/syntax/methods_rdoc.html#label-Block+Argument)
docs.ruby-lang.org/en/4.0/syntax/methods_rdoc.html#label-Block+Argument
- [Enumerable usage](https://docs.ruby-lang.org/en/4.0/Enumerable.html#module-Enumerable-label-Usage)
docs.ruby-lang.org/en/4.0/Enumerable.html#module-Enumerable-label-Usage
- [Enumerable#count](https://docs.ruby-lang.org/en/4.0/Enumerable.html#method-i-count)
docs.ruby-lang.org/en/4.0/Enumerable.html#method-i-count
- [Enumerable#any?](https://docs.ruby-lang.org/en/4.0/Enumerable.html#method-i-any?)
docs.ruby-lang.org/en/4.0/Enumerable.html#method-i-any-3F
- [Symbol#to_proc](https://docs.ruby-lang.org/en/4.0/Symbol.html#method-i-to_proc)
docs.ruby-lang.org/en/4.0/Symbol.html#method-i-to_proc
- [Proc to block conversion](https://docs.ruby-lang.org/en/4.0/syntax/calling_methods_rdoc.html#label-Proc+to+Block+Conversion)
docs.ruby-lang.org/en/4.0/syntax/calling_methods_rdoc.html#label-Proc+to+Block+Conversion
- [Lambda and non-lambda semantics](https://docs.ruby-lang.org/en/4.0/Proc.html#class-Proc-label-Lambda+and+non-lambda+semantics)
docs.ruby-lang.org/en/4.0/Proc.html#class-Proc-label-Lambda+and+non-lambda+semantics

Exercice 5 - Gestion des erreurs et exceptions

- [Kernel#raise](https://docs.ruby-lang.org/en/4.0/Kernel.html#method-i-raise)
docs.ruby-lang.org/en/4.0/Kernel.html#method-i-raise
- [Custom exceptions](https://docs.ruby-lang.org/en/4.0/language/exceptions_md.html#label-Custom+Exceptions)
docs.ruby-lang.org/en/4.0/language/exceptions_md.html#label-Custom+Exceptions
- [Exception.new](https://docs.ruby-lang.org/en/4.0/Exception.html#method-c-new)
docs.ruby-lang.org/en/4.0/Exception.html#method-c-new
- [Exception#message](https://docs.ruby-lang.org/en/4.0/Exception.html#method-i-message)
docs.ruby-lang.org/en/4.0/Exception.html#method-i-message
- [Rescue clauses](https://docs.ruby-lang.org/en/4.0/language/exceptions_md.html#label-Rescue+Clauses)
docs.ruby-lang.org/en/4.0/language/exceptions_md.html#label-Rescue+Clauses
- [Multiple rescue clauses](https://docs.ruby-lang.org/en/4.0/language/exceptions_md.html#label-Multiple+Rescue+Clauses)
docs.ruby-lang.org/en/4.0/language/exceptions_md.html#label-Multiple+Rescue+Clauses

- [Built-in exception class hierarchy](https://docs.ruby-lang.org/en/4.0/Exception.html#class-Exception-label-Built-In+Exception+Class+Hierarchy)
docs.ruby-lang.org/en/4.0/Exception.html#class-Exception-label-Built-In+Exception+Class+Hierarchy
- [Else clause](https://docs.ruby-lang.org/en/4.0/language/exceptions_md.html#label-Else+Clause)
docs.ruby-lang.org/en/4.0/language/exceptions_md.html#label-Else+Clause
- [Ensure clause](https://docs.ruby-lang.org/en/4.0/language/exceptions_md.html#label-Ensure+Clause)
docs.ruby-lang.org/en/4.0/language/exceptions_md.html#label-Ensure+Clause
- [Re-raising an exception](https://docs.ruby-lang.org/en/4.0/language/exceptions_md.html#label-Re-Raising+an+Exception)
docs.ruby-lang.org/en/4.0/language/exceptions_md.html#label-Re-Raising+an+Exception
- [Begin-less exception handlers](https://docs.ruby-lang.org/en/4.0/language/exceptions_md.html#label-Begin-Less+Exception+Handlers)
docs.ruby-lang.org/en/4.0/language/exceptions_md.html#label-Begin-Less+Exception+Handlers

Exercise 6 - Métaprogrammation et introspection

- [Module#prepend](https://docs.ruby-lang.org/en/4.0/Module.html#method-i-prepend)
docs.ruby-lang.org/en/4.0/Module.html#method-i-prepend
- [BasicObject#method_missing](https://docs.ruby-lang.org/en/4.0/BasicObject.html#method-i-method_missing)
docs.ruby-lang.org/en/4.0/BasicObject.html#method-i-method_missing
- [Method#source_location](https://docs.ruby-lang.org/en/4.0/Method.html#method-i-source_location)
docs.ruby-lang.org/en/4.0/Method.html#method-i-source_location
- [Kernel#caller](https://docs.ruby-lang.org/en/4.0/Kernel.html#method-i-caller)
docs.ruby-lang.org/en/4.0/Kernel.html#method-i-caller
- [Kernel#binding](https://docs.ruby-lang.org/en/4.0/Kernel.html#method-i-binding)
docs.ruby-lang.org/en/4.0/Kernel.html#method-i-binding
- [Object#send](https://docs.ruby-lang.org/en/4.0/Object.html#method-i-send)
docs.ruby-lang.org/en/4.0/Object.html#method-i-send
- [Object#public_send](https://docs.ruby-lang.org/en/4.0/Object.html#method-i-public_send)
docs.ruby-lang.org/en/4.0/Object.html#method-i-public_send
- [Kernel#puts](https://docs.ruby-lang.org/en/4.0/Kernel.html#method-i-puts)
docs.ruby-lang.org/en/4.0/Kernel.html#method-i-puts
- [Object#respond_to_missing?](https://docs.ruby-lang.org/en/4.0/Object.html#method-i-respond_to_missing-3F)
docs.ruby-lang.org/en/4.0/Object.html#method-i-respond_to_missing-3F
- [Module#define_method](https://docs.ruby-lang.org/en/4.0/Module.html#method-i-define_method)
docs.ruby-lang.org/en/4.0/Module.html#method-i-define_method
- [Object#instance_variable_get](https://docs.ruby-lang.org/en/4.0/Object.html#method-i-instance_variable_get)
docs.ruby-lang.org/en/4.0/Object.html#method-i-instance_variable_get
- [Object#instance_variable_set](https://docs.ruby-lang.org/en/4.0/Object.html#method-i-instance_variable_set)
docs.ruby-lang.org/en/4.0/Object.html#method-i-instance_variable_set
- [Module#included](https://docs.ruby-lang.org/en/4.0/Module.html#method-i-included)
docs.ruby-lang.org/en/4.0/Module.html#method-i-included
- [Module#class_eval](https://docs.ruby-lang.org/en/4.0/Module.html#method-i-class_eval)
docs.ruby-lang.org/en/4.0/Module.html#method-i-class_eval
- [Overriding](https://docs.ruby-lang.org/en/4.0/syntax/methods_rdoc.html#label-Overriding)
docs.ruby-lang.org/en/4.0/syntax/methods_rdoc.html#label-Overriding
- [String#unicode_normalize](https://docs.ruby-lang.org/en/4.0/String.html#method-i-unicode_normalize)
docs.ruby-lang.org/en/4.0/String.html#method-i-unicode_normalize

Exercise 7 - Synthèse (bonus)

- [Module.new](https://docs.ruby-lang.org/en/4.0/Module.html#method-c-new)
docs.ruby-lang.org/en/4.0/Module.html#method-c-new
- [Kernel#require_relative](https://docs.ruby-lang.org/en/4.0/Kernel.html#method-i-require_relative)
docs.ruby-lang.org/en/4.0/Kernel.html#method-i-require_relative